

Introduction to Python Programming

Chapter 10: File Input/Output

Introduction

So far, your programs lose all their data when they close. Want to save user information? Store game scores? Keep configuration settings? You need **file I/O** (Input/Output). This chapter teaches you how to read from files, write to files, and handle the common errors that occur during file operations. Combined with exception handling from the previous chapter, you'll create robust programs that work with persistent data.

1. Understanding File Operations

What is File I/O?

File I/O is reading data from files and writing data to files. It's how programs save and retrieve information that persists between runs.

Three basic operations:

1. **Opening** a file - getting access to it
2. **Reading/Writing** - getting data from it or putting data into it
3. **Closing** a file - releasing it when done

File Paths

Before working with files, you need to know where they are:

Absolute paths (full path from root):

```
# Windows
"C:\\Users\\Alice\\Documents\\data.txt"

# macOS/Linux
"/home/alice/documents/data.txt"
```

Relative paths (from current directory):

```
"data.txt"           # Same directory as your Python file
"files/data.txt"     # In a 'files' subdirectory
"../data.txt"        # Parent directory
```

Best practice: Use relative paths for portability

File Modes

When opening a file, specify the **mode**:

Mode	Description	Creates if Missing?	Truncates Existing?
'r'	Read (default)	No (raises error)	No
'w'	Write	Yes	Yes
'a'	Append	Yes	No
'r+'	Read + Write	No (raises error)	No
'w+'	Write + Read	Yes	Yes
'a+'	Append + Read	Yes	No

Add 'b' for binary mode: 'rb', 'wb', etc.

2. Reading Files

Opening and Reading Files

Basic File Reading

```
# Open file
file = open("data.txt", "r")

# Read entire file
content = file.read()
print(content)

# Close file (important!)
file.close()
```

Problems with this approach:

- If error occurs, file might not close
- Manual cleanup required

Better: Using `with` Statement

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# File automatically closes when exiting the 'with' block
```

Why `with` is better:

- Automatically closes file (even if error occurs)
- No need to remember `file.close()`
- Best practice for file operations

Reading Methods

read() - Read Entire File

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

readline() - Read One Line

```
with open("data.txt", "r") as file:
    line1 = file.readline()
    line2 = file.readline()
    print(line1)
    print(line2)
```

readlines() - Read All Lines into List

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    # Returns list: ['line 1\n', 'line 2\n', 'line 3\n']

for line in lines:
    print(line.strip()) # strip() removes \n
```

Iterating Over File (Best for Large Files)

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

This is memory efficient - reads one line at a time, not all at once.

Reading with Exception Handling

```
try:
    with open("data.txt", "r") as file:
        content = file.read()
        print(content)
except FileNotFoundError:
    print("Error: File 'data.txt' not found!")
except PermissionError:
    print("Error: No permission to read this file!")
except Exception as e:
    print(f"Unexpected error: {e}")
```

3. Writing to Files

Basic Writing

write() - Write String to File

```
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("This is a new file.\n")
```

Warning: Mode `'w'` **overwrites** existing files!

writelines() - Write List of Strings

```
lines = ["First line\n", "Second line\n", "Third line\n"]

with open("output.txt", "w") as file:
    file.writelines(lines)
```

Appending to Files

Use mode `'a'` to add to existing file without overwriting:

```
with open("log.txt", "a") as file:
    file.write("New log entry\n")
```

Writing with Exception Handling

```

try:
    with open("output.txt", "w") as file:
        file.write("Important data\n")
        print("File written successfully!")
except PermissionError:
    print("Error: No permission to write to this file!")
except IOError as e:
    print(f"Error writing to file: {e}")

```

4. Practical File Operations

Example 1: Reading Configuration File

```

def load_config(filename):
    """Load configuration from file."""
    config = {}

    try:
        with open(filename, "r") as file:
            for line in file:
                line = line.strip()
                if line and '=' in line:
                    key, value = line.split('=', 1)
                    config[key.strip()] = value.strip()
    except FileNotFoundError:
        print(f"Config file '{filename}' not found. Using defaults.")

    return config

# config.txt contents:
# username=alice
# theme=dark
# font_size=14

config = load_config("config.txt")
print(config)  # {'username': 'alice', 'theme': 'dark', 'font_size': '14'}

```

Example 2: Writing Log File

```

from datetime import datetime

def log_message(message, filename="app.log"):
    """Append timestamped message to log file."""
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}\n"

```

```

try:
    with open(filename, "a") as file:
        file.write(log_entry)
except IOError as e:
    print(f"Failed to write to log: {e}")

log_message("Application started")
log_message("User logged in")
log_message("Error: Failed to connect to database")

```

Example 3: Reading CSV-Like Data

```

def read_student_data(filename):
    """Read student data from CSV-like file."""
    students = []

    try:
        with open(filename, "r") as file:
            for line in file:
                # Skip empty lines or headers
                line = line.strip()
                if not line or line.startswith("#"):
                    continue

                # Split by comma
                parts = line.split(",")
                if len(parts) == 3:
                    name, age, grade = parts
                    students.append({
                        "name": name.strip(),
                        "age": int(age.strip()),
                        "grade": grade.strip()
                    })
    except FileNotFoundError:
        print(f"File '{filename}' not found!")
    except ValueError as e:
        print(f"Error parsing data: {e}")

    return students

# students.csv contents:
# Alice,20,A
# Bob,21,B
# Charlie,19,A

students = read_student_data("students.csv")
for student in students:

```

```
print(f"{student['name']} - Age: {student['age']}, Grade: {student['grade']}")
```

Example 4: Saving High Scores

```
def save_high_scores(scores, filename="highscores.txt"):
    """Save high scores to file."""
    try:
        with open(filename, "w") as file:
            for i, score in enumerate(scores, 1):
                file.write(f"{i}. {score['name']}: {score['points']}\n")
            print(f"High scores saved to {filename}")
    except IOError as e:
        print(f"Error saving scores: {e}")

def load_high_scores(filename="highscores.txt"):
    """Load high scores from file."""
    scores = []

    try:
        with open(filename, "r") as file:
            for line in file:
                # Parse: "1. Alice: 1000"
                if ". " in line and ": " in line:
                    parts = line.split(". ", 1)[1] # Remove number
                    name, points = parts.split(": ")
                    scores.append({
                        "name": name.strip(),
                        "points": int(points.strip())
                    })
    except FileNotFoundError:
        print("No high scores file found. Starting fresh.")
    except ValueError as e:
        print(f"Error parsing scores: {e}")

    return scores

# Usage
high_scores = [
    {"name": "Alice", "points": 1000},
    {"name": "Bob", "points": 850},
    {"name": "Charlie", "points": 900}
]
save_high_scores(high_scores)

loaded_scores = load_high_scores()
print(loaded_scores)
```

5. Working with File Paths

Using os.path Module

```
import os

# Check if file exists
if os.path.exists("data.txt"):
    print("File exists!")

# Check if it's a file (not directory)
if os.path.isfile("data.txt"):
    print("It's a file!")

# Check if it's a directory
if os.path.isdir("my_folder"):
    print("It's a directory!")

# Get absolute path
abs_path = os.path.abspath("data.txt")
print(f"Absolute path: {abs_path}")

# Join paths (works on all OS)
file_path = os.path.join("folder", "subfolder", "file.txt")
print(file_path) # folder/subfolder/file.txt (or folder\subfolder\file.txt on Windows)

# Split path into directory and filename
directory, filename = os.path.split("/path/to/file.txt")
print(f"Directory: {directory}") # /path/to
print(f"Filename: {filename}") # file.txt

# Get file extension
name, ext = os.path.splitext("document.txt")
print(f"Name: {name}, Extension: {ext}") # Name: document, Extension: .txt
```

Creating Directories

```
import os

# Create single directory
if not os.path.exists("data"):
    os.mkdir("data")

# Create nested directories
if not os.path.exists("data/reports/2024"):
    os.makedirs("data/reports/2024")
```


Listing Files

```
import os

# List all files in directory
files = os.listdir(".")
for file in files:
    print(file)

# List only .txt files
for file in os.listdir("."):
    if file.endswith(".txt"):
        print(file)
```

6. Binary Files

Reading Binary Files

```
# Read image file
with open("image.jpg", "rb") as file:
    data = file.read()
    print(f"File size: {len(data)} bytes")
```

Writing Binary Files

```
# Copy a file
with open("source.jpg", "rb") as source:
    data = source.read()

with open("destination.jpg", "wb") as destination:
    destination.write(data)

print("File copied successfully!")
```

Practical Example: File Copy Utility

```
def copy_file(source, destination):
    """Copy file from source to destination."""
    try:
        with open(source, "rb") as src:
            data = src.read()

        with open(destination, "wb") as dst:
```

```

        dst.write(data)

    print(f"Copied {source} to {destination}")
    return True
except FileNotFoundError:
    print(f"Error: Source file '{source}' not found!")
    return False
except PermissionError:
    print("Error: Permission denied!")
    return False
except Exception as e:
    print(f"Error copying file: {e}")
    return False

copy_file("document.pdf", "backup/document.pdf")

```

7. Working with JSON Files

JSON is perfect for saving structured data:

```

import json

# Save data to JSON
data = {
    "name": "Alice",
    "age": 25,
    "scores": [85, 90, 92],
    "is_active": True
}

with open("data.json", "w") as file:
    json.dump(data, file, indent=4)

# Load data from JSON
with open("data.json", "r") as file:
    loaded_data = json.load(file)
    print(loaded_data)

```

Practical Example: User Settings

```

import json

def save_settings(settings, filename="settings.json"):
    """Save user settings to JSON file."""
    try:
        with open(filename, "w") as file:

```

```

        json.dump(settings, file, indent=4)
    print("Settings saved successfully!")
except IOError as e:
    print(f"Error saving settings: {e}")

def load_settings(filename="settings.json"):
    """Load user settings from JSON file."""
    default_settings = {
        "theme": "light",
        "language": "en",
        "notifications": True
    }

    try:
        with open(filename, "r") as file:
            return json.load(file)
    except FileNotFoundError:
        print("No settings file found. Using defaults.")
        return default_settings
    except json.JSONDecodeError:
        print("Error: Invalid JSON in settings file. Using defaults.")
        return default_settings

# Usage
settings = load_settings()
settings["theme"] = "dark"
save_settings(settings)

```

8. Context Managers and File Safety

Why Use `with` Statement

The `with` statement is a **context manager** that ensures proper cleanup:

```

# Without 'with' - risky
file = open("data.txt", "r")
content = file.read() # If error occurs here...
file.close()          # ...this might not run!

# With 'with' - safe
with open("data.txt", "r") as file:
    content = file.read() # If error occurs, file still closes

```

Multiple Files

```
# Open multiple files safely
with open("input.txt", "r") as infile, open("output.txt", "w") as outfile:
    for line in infile:
        outfile.write(line.upper())
```

9. Common File Operations Patterns

Pattern 1: Read, Process, Write

```
def process_file(input_file, output_file):
    """Read file, process each line, write result."""
    try:
        with open(input_file, "r") as infile:
            lines = infile.readlines()

        processed_lines = [line.upper() for line in lines]

        with open(output_file, "w") as outfile:
            outfile.writelines(processed_lines)

        print(f"Processed {len(lines)} lines")
    except FileNotFoundError:
        print(f"Error: '{input_file}' not found!")
    except IOError as e:
        print(f"Error: {e}")

process_file("input.txt", "output.txt")
```

Pattern 2: Append to Log

```
def append_to_log(message, log_file="activity.log"):
    """Append message to log file with timestamp."""
    from datetime import datetime

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}\n"

    try:
        with open(log_file, "a") as file:
            file.write(log_entry)
    except IOError as e:
        print(f"Failed to log: {e}")

append_to_log("User login successful")
```

Pattern 3: Safe File Update

```
def update_file_safely(filename, new_content):
    """Update file with backup."""
    import shutil

    backup_file = filename + ".backup"

    try:
        # Create backup if file exists
        if os.path.exists(filename):
            shutil.copy(filename, backup_file)

        # Write new content
        with open(filename, "w") as file:
            file.write(new_content)

        print("File updated successfully!")

        # Remove backup if successful
        if os.path.exists(backup_file):
            os.remove(backup_file)

    except Exception as e:
        print(f"Error updating file: {e}")
        # Restore backup if it exists
        if os.path.exists(backup_file):
            shutil.copy(backup_file, filename)
            print("Restored from backup")
```

Pattern 4: Read Until Condition

```
def read_until_marker(filename, marker="---END---"):
    """Read file until marker is found."""
    content = []

    try:
        with open(filename, "r") as file:
            for line in file:
                if marker in line:
                    break
                content.append(line)
    except FileNotFoundError:
        print(f"File '{filename}' not found!")

    return "".join(content)
```

10. Best Practices

Do's

✓ Always use `with` statement:

```
with open("file.txt", "r") as file:
    content = file.read()
```

✓ Handle exceptions:

```
try:
    with open("file.txt", "r") as file:
        content = file.read()
except FileNotFoundError:
    print("File not found!")
```

✓ Use appropriate modes:

```
# Read existing file
with open("data.txt", "r") as file:
    ...

# Create new or overwrite
with open("output.txt", "w") as file:
    ...

# Add to existing
with open("log.txt", "a") as file:
    ...
```

✓ Use `os.path` for path operations:

```
import os
path = os.path.join("folder", "file.txt")
```

✓ Close files (or use `with`):

```
# If not using 'with', always close
file = open("data.txt", "r")
try:
    content = file.read()
finally:
    file.close()
```

Don'ts

✗ Don't forget to close files:

```
# Bad
file = open("data.txt", "r")
content = file.read()
# Forgot file.close()!
```

✗ Don't ignore errors:

```
# Bad
with open("data.txt", "r") as file:
    content = file.read()
# What if file doesn't exist?
```

✗ Don't use 'w' without checking:

```
# Bad - overwrites existing file!
with open("important.txt", "w") as file:
    file.write("Oops!")
```

✗ Don't hardcode paths:

```
# Bad
file = open("C:\\Users\\Alice\\data.txt") # Won't work for other users

# Good
import os
file_path = os.path.join(os.getcwd(), "data.txt")
```

Key Takeaways

- ✓ Use `with` statement for automatic file closing
- ✓ **File modes:** 'r' (read), 'w' (write/overwrite), 'a' (append)
- ✓ **Reading methods:** `read()`, `readline()`, `readlines()`, or `iterate`
- ✓ **Always handle exceptions** - `FileNotFoundError`, `PermissionError`, `IOError`
- ✓ Use `os.path` for cross-platform path operations
- ✓ **JSON** is great for structured data
- ✓ **Binary mode ('rb', 'wb')** for images, videos, etc.

- ✓ **Create backups** before overwriting important files
 - ✓ **Close files** to release resources (automatic with `with`)
 - ✓ **Check file existence** before operations when appropriate
-

Reflection Questions

1. What's the difference between 'r', 'w', and 'a' modes?
 2. Why is the `with` statement better than manually closing files?
 3. What exception occurs when a file doesn't exist?
 4. How do you read a file line by line efficiently?
 5. What's the difference between `write()` and `writelines()`?
 6. How do you check if a file exists before opening it?
 7. When should you use binary mode ('rb', 'wb')?
 8. What's the advantage of using JSON for data storage?
-

Practice Exercises

1. **File Reader:** Write a program that reads a text file and displays its contents
2. **Line Counter:** Count the number of lines in a text file
3. **Word Counter:** Count total words in a text file
4. **File Copy:** Copy contents from one file to another
5. **Search and Replace:** Read file, replace all occurrences of a word, save to new file
6. **Grade Manager:** Save student grades to file, load them back, calculate averages
7. **Log Analyzer:** Read a log file and count occurrences of "ERROR" vs "INFO"
8. **Configuration Manager:** Save/load program settings using JSON
9. **High Score System:** Save and display top 10 high scores
10. **Text Statistics:** Read file and report: lines, words, characters, average word length